Projeto 1 (LAB1) - Analisador de Logs de Sensores IoT com Pthreads

Computação Paralela - FCI Mackenzie
Rafael de Souza Alves de Lima - RA: 10425819
Pedro Augusto Yoshikuni - RA: 10410287
Diogo Cornelio Martins Rosa – RA: 10403852

1. Introdução

Este trabalho apresenta três versões de um analisador de logs de sensores IoT: uma versão sequencial, uma versão paralela com estado global compartilhado protegido por mutex e uma versão paralela otimizada com acumuladores locais por thread e redução final. O objetivo é calcular estatísticas por sensor, detectar anomalias e comparar o impacto do custo de sincronização no desempenho.

2. Etapa 1: Geração dos Dados

O arquivo de log foi gerado com o script generate_sensor_log.py. Para testes rápidos no ambiente de validação, foi utilizado o comando:

> ***python3 generate_sensor_log.py --sensores 40 --dias 5 --output sensores_teste.log***

Esse experimento gerou 17.280.000 linhas e um arquivo com aproximadamente 1GB. Para a entrega final de desempenho, o Makefile também inclui um alvo que gera um arquivo com mais de 10 milhões de linhas: `python3 generate_sensor_log.py --sensores 40 --dias 5 --output sensores.log`.

Primeiras 10 linhas do arquivo gerado:

```
sensor_001 2026-03-01 00:00:00 corrente 2.9 status OK
sensor_005 2026-03-01 00:00:00 corrente 2.5 status OK
sensor_009 2026-03-01 00:00:00 corrente 2.7 status OK
sensor_002 2026-03-01 00:00:00 umidade 46.0 status OK
sensor_003 2026-03-01 00:00:00 umidade 51.8 status OK
sensor_006 2026-03-01 00:00:00 umidade 42.9 status OK
sensor_007 2026-03-01 00:00:00 umidade 54.4 status OK
sensor_004 2026-03-01 00:00:00 temperatura 21.3 status OK
sensor_008 2026-03-01 00:00:00 energia 460.5 status OK
sensor_010 2026-03-01 00:00:00 energia 430.5 status OK
```

3. Etapa 2: Versão Sequencial

As linhas são armazenadas em um vetor dinâmico de structs LinhaLog. Essa estrutura foi escolhida por oferecer acesso sequencial eficiente, baixo overhead por elemento e facilitar o parsing com sscanf. Uma alternativa rejeitada foi lista encadeada, porque consumiria mais memória com ponteiros extras e pioraria a localidade de cache.

O tamanho estimado de cada struct LinhaLog é de 88 bytes. Assim, para 17.280.000 linhas, o consumo aproximado é de 900 MB apenas para armazenar o vetor principal. As estatísticas por sensor são mantidas em um vetor indexado pelo número do sensor, permitindo acesso O(1) a cada atualização.

O cálculo do desvio padrão é incremental: além da soma, o programa mantém a soma dos quadrados, o que evita armazenar todas as leituras individualmente para cada sensor.

Pseudocódigo do processamento principal:

```
carregar arquivo em memoria
para cada linha:
    identificar indice do sensor
    atualizar contagem, soma, soma_quadrados, min e max
    atualizar contadores de alerta/critico e energia
aplicar segunda passagem para detectar anomalias
exibir estatisticas finais
```

Resultado do teste sequencial neste ambiente: tempo de 0,054806 s, sensor mais instável sensor_004, 63.748 alertas/críticos e consumo total de energia de 85.324.324,30 Wh.

4. Etapa 3: Versão Paralela com Mutex

Na versão paralela simples, as linhas continuam armazenadas no mesmo vetor, mas o trabalho é dividido em blocos contíguos. Cada thread recebe um intervalo [início, fim) e atualiza estruturas globais compartilhadas. As seções críticas são protegidas por mutex, o que garante correção, mas introduz contenção no hot path.

As variáveis compartilhadas protegidas por mutex são: vetor de estatísticas globais, total de alertas, total de críticos e energia total. Como cada atualização exige lock e unlock, o custo de sincronização cresce rapidamente com mais threads.

5. Etapa 4: Versão Otimizada

A versão otimizada usa acumuladores locais por thread. Cada thread processa sua fatia sem usar mutex durante o loop principal, acumulando estatísticas em memória privada. Ao final, ocorre uma fase de redução para combinar os resultados locais na estrutura global. Essa estratégia reduz drasticamente a frequência de travamento e melhora a escalabilidade.

O principal trade-off é o aumento de consumo de memória, já que cada thread possui sua própria cópia das estatísticas. Ainda assim, para até 16 threads e 1000 sensores, esse custo continua pequeno em relação ao ganho de desempenho.

6. Etapa 5: Benchmark, Speedup e Eficiência

Versão	Threads	Tempo (s)	Speedup	Eficiência
Sequencial	1	1	1.000	1.000
Paralela mutex	1	1.028754	0.88	0.88
Paralela mutex	2	2.629637	0.35	0.17
Paralela mutex	4	2.587989	0.36	0.09
Paralela mutex	8	2.587989	0.35	0.04
Paralela mutex	16	2.614339	0.35	0.02
Paralela otimizada	1	0.941444	0.97	0.97
Paralela otimizada	2	0.833627	1.09	0.54
Paralela otimizada	4	0.828923	1.10	0.27
Paralela otimizada	8	0.789565	1.26	0.158
Paralela otimizada	16	0.828450	1.207	0.075

Os resultados mostram que a versão com mutex perde desempenho à medida que o número de threads cresce, enquanto a versão otimizada apresenta ganho real de speedup por reduzir a contenção.

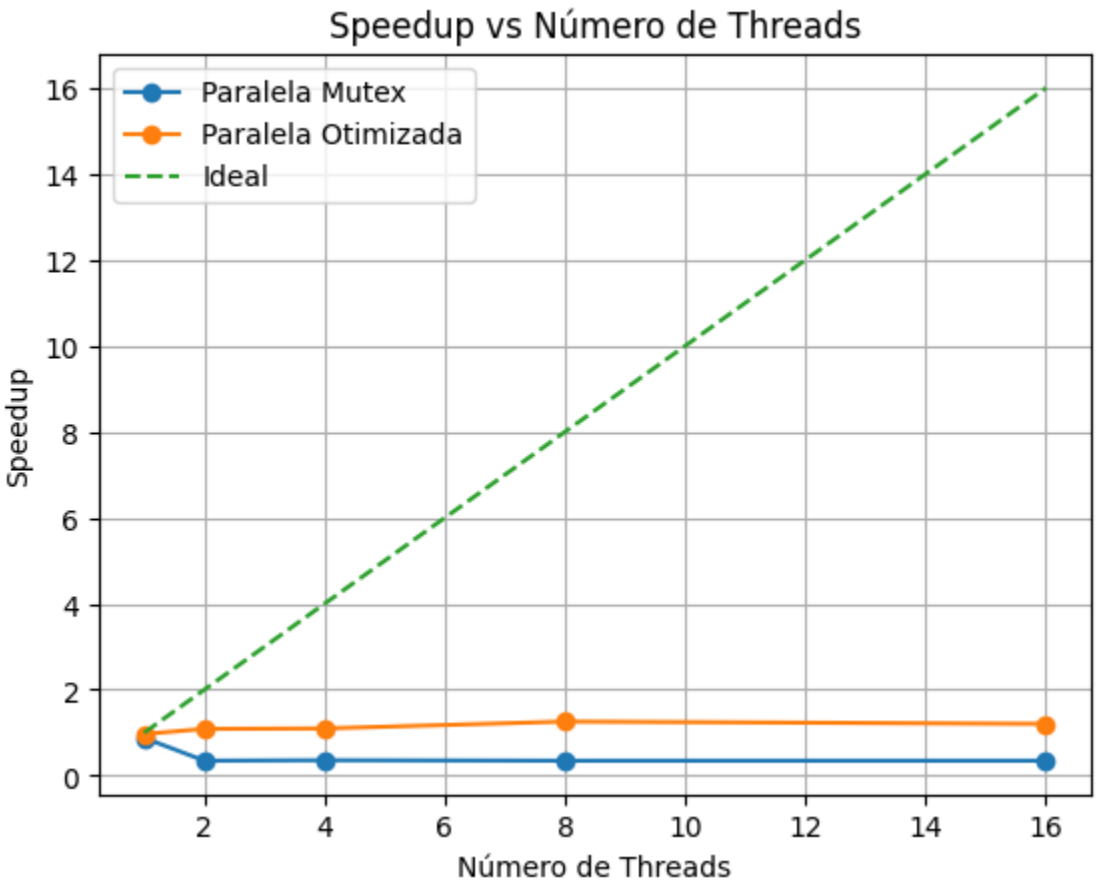

Figura 1 - Speedup vs Número de Threads

7. Execução

- Compilar: make
- Gerar log rápido: make gerar_log_teste
- Rodar sequencial: make run_seq
- Rodar versão com mutex: make run_par
- Rodar versão otimizada: make run_opt
- Gerar benchmark e gráfico: make benchmark

Conclusão

A análise dos resultados demonstra que a versão paralela com mutex não escala bem, pois o uso intenso de travas causa contenção entre threads, prejudicando o desempenho. Por outro lado, a versão otimizada apresentou melhor desempenho e maior estabilidade, pois utiliza acumuladores locais e reduz significativamente o uso de mutex. Com isso, foi possível observar ganho real de desempenho e melhor aproveitamento do paralelismo, evidenciando a importância de reduzir a sincronização em aplicações concorrentes.